openai / evals Public

Code | Issues 121 | Pull requests 68 | Discussions | Actions

89 Branches 10 Tags Go to file Go to file Code

mcgrew-oai and codex ... k revisions to immutable ommit...

8eac7a7 · 2 y...

	s w...	3 weeks ago
	...	2 years ago
	.	5 months ago
	.	3 years ago
	...	3 years ago
	ro...	3 years ago
.gita		3 years ago
.gitignore	g eval (#1401)	3 years ago
.pre-commit-config.ya...	Pin pre-commit hook revision...	2 weeks ago
LICENSE.md	Already Said That Eval (#1490)	2 years ago
MANIFEST.in	Keep data pull from main bra...	3 years ago
Makefile	Makefile: Set `mypy` as a phony...	3 years ago
README.md	Updating readme to link to O...	2 years ago
SECURITY.md	Add download command for ...	3 years ago
mypy.ini	Correct the types of registry ...	3 years ago
pyproject.toml	Release 3.0.1 (#1525)	2 years ago

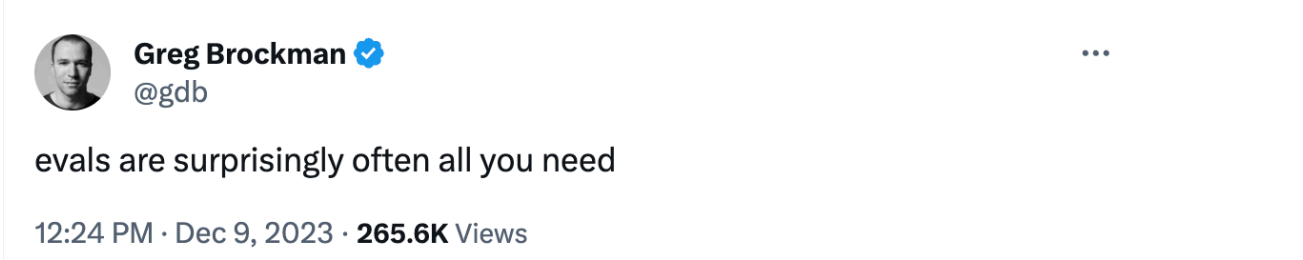
README | License | Security

OpenAI Evals

> You can now configure and run Evals directly in the OpenAI Dashboard. Get started →

Evals provide a framework for evaluating large language models (LLMs) or systems built using LLMs. We offer an existing registry of evals to test different dimensions of OpenAI models and the ability to write your own custom evals for use cases you care about. You can also use your data to build private evals which represent the common LLMs patterns in your workflow without exposing any of that data publicly.

If you are building with LLMs, creating high quality evals is one of the most impactful things you can do. Without evals, it can be very difficult and time intensive to understand how different model versions might affect your use case. In the words of OpenAI's President Greg Brockman:



Setup

To run evals, you will need to set up and specify your OpenAI API key. After you obtain an API key, specify it using the `OPENAI_API_KEY` environment variable. Please be aware of the costs associated with using the API when running evals. You can also run and create evals using Weights & Biases.

Minimum Required Version: Python 3.9

Downloading evals

Our evals registry is stored using Git-LFS. Once you have downloaded and installed LFS, you can fetch the evals (from within your local copy of the evals repo) with:

```
cd evals
git lfs fetch --all
git lfs pull
```

This will populate all the pointer files under `evals/registry/data`.

You may just want to fetch data for a select eval. You can achieve this via:

```
git lfs fetch --include=evals/registry/data/${your eval}
git lfs pull
```

Making evals

If you are going to be creating evals, we suggest cloning this repo directly from GitHub and installing the requirements using the following command:

```
pip install -e .
```

Using `-e`, changes you make to your eval will be reflected immediately without having to reinstall.

Optionally, you can install the formatters for pre-committing with:

```
pip install -e .[formatters]
```

Then run `pre-commit install` to install pre-commit into your git hooks. pre-commit will now run on every commit.

If you want to manually run all pre-commit hooks on a repository, run `pre-commit run --all-files`. To run individual hooks use `pre-commit run <hook_id>`.

Running evals

If you don't want to contribute new evals, but simply want to run them locally, you can install the evals package via pip:

```
pip install evals
```

You can find the full instructions to run existing evals in `run-evals.md` and our existing eval templates in `eval-templates.md`. For more advanced use cases like prompt chains or tool-using agents, you can use our Completion Function Protocol.

We provide the option for you to log your eval results to a Snowflake database, if you have one or wish to set one up. For this option, you will further have to specify the `SNOWFLAKE_ACCOUNT`, `SNOWFLAKE_DATABASE`, `SNOWFLAKE_USERNAME`, and `SNOWFLAKE_PASSWORD` environment variables.

Writing evals

We suggest getting starting by:

- Walking through the process for building an eval: `build-eval.md`
- Exploring an example of implementing custom eval logic: `custom-eval.md`
- Writing your own completion functions: `completion-fns.md`
- Review our starter guide for writing evals: Getting Started with OpenAI Evals

Please note that we are currently not accepting evals with custom code! While we ask you to not submit such evals at the moment, you can still submit model-graded evals with custom model-graded YAML files.

If you think you have an interesting eval, please open a pull request with your contribution. OpenAI staff actively review these evals when considering improvements to upcoming models.

FAQ

Do you have any examples of how to build an eval from start to finish?

- Yes! These are in the `examples` folder. We recommend that you also read through `build-eval.md` in order to gain a deeper understanding of what is happening in these examples.

Do you have any examples of evals implemented in multiple different ways?

- Yes! In particular, see `evals/registry/evals/coqa.yaml`. We have implemented small subsets of the CoQA dataset for various eval templates to help illustrate the differences.

When I run an eval, it sometimes hangs at the very end (after the final report). What's going on?

- This is a known issue, but you should be able to interrupt it safely and the eval should finish immediately after.

There's a lot of code, and I just want to spin up a quick eval. Help? OR,

I am a world-class prompt engineer. I choose not to code. How can I contribute my wisdom?

- If you follow an existing eval template to build a basic or model-graded eval, you don't need to write any evaluation code at all! Just provide your data in JSON format and specify your eval parameters in YAML. build-eval.md walks you through these steps, and you can supplement these instructions with the Jupyter notebooks in the `examples`

folder to help you get started quickly. Keep in mind, though, that a good eval will inevitably require careful thought and rigorous experimentation!

Disclaimer

By contributing to evals, you are agreeing to make your evaluation logic and data under the same MIT license as this repository. You must have adequate rights to upload any data used in an eval. OpenAI reserves the right to use this data in future service improvements to our product. Contributions to OpenAI evals will be subject to our usual Usage Policies.

Contributors 462

+ 448 contributors

Languages

- Python 89.4%
- HTML 6.0%
- Jupyter Notebook 2.4%
- Shell 1.8%
- JavaScript 0.4%
- Dockerfile 0.0%